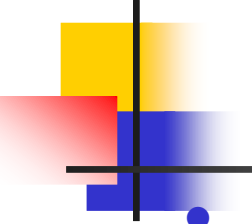




Ch4 内核与驱动简介

Enyi Tang
Software Institute of
Nanjing University

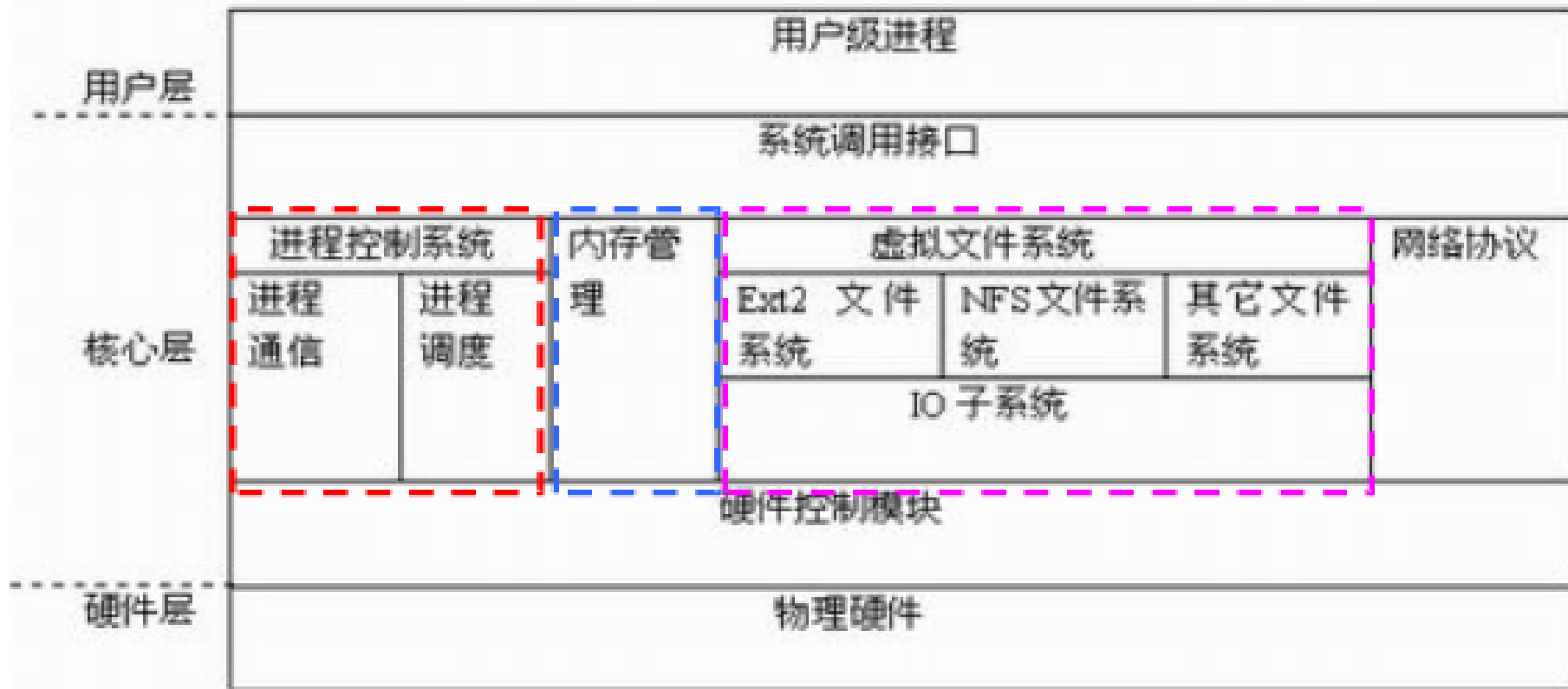


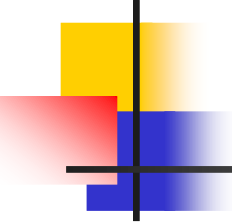
Linux内核简介

- 什么是内核

- 操作系统是一系列程序的集合，其中最重要的部分构成了内核
- 单内核/微内核
 - 单内核是一个很大的进程，内部可以分为若干模块，运行时是一个独立的二进制文件，模块间通讯通过直接调用函数实现
 - 微内核中大部分内核作为独立的进程在特权下运行，通过消息传递进行通讯
- Linux内核的能力
 - 内存管理，文件系统，进程管理，多线程支持，抢占式，多处理支持
- Linux内核区别于其他UNIX商业内核的优点
 - 单内核，模块支持
 - 免费/开源
 - 支持多种CPU，硬件支持能力非常强大
 - Linux开发者都是非常出色的程序员
 - 通过学习Linux内核的源码可以了解现代操作系统的实现原理

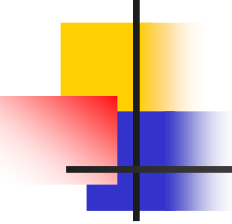
层次结构





内核源代码获取

- <https://www.kernel.org/>
- apt-get方式
 - apt-cache search linux-source //查看内核版本
 - apt-get install linux-source-3.2
 - 下载下来的位置一般在/usr/src
- 从Ubuntu的源码库中获得内核源码
 - git clone
git://kernel.ubuntu.com/ubuntu/ubuntu-hardy.git



后续操作

- 解压
 - `tar jxvf /home/ldd/linux-3.2.tar.bz2`
- 清除先前编译产生的目标文件
 - `make clean`
- 配置内核
 - `make menuconfig`

File Edit View Search Terminal Help

.config - Linux/x86_64 3.2.54 Kernel Configuration**Linux/x86_64 3.2.54 Kernel Configuration**

Arrow keys navigate the menu. <Enter> selects submenus --->.
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes,
<M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </>
for Search. Legend: [*] built-in [] excluded <M> module < >

General setup --->

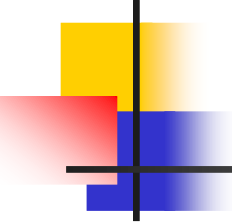
- [*] Enable loadable module support --->
- *- Enable the block layer --->
 - Processor type and features --->
 - Power management and ACPI options --->
 - Bus options (PCI etc.) --->
 - Executable file formats / Emulations --->
- *- Networking support --->
 - Device Drivers --->
 - Firmware Drivers --->

v(+)

<Select>

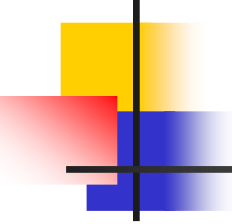
< Exit >

< Help >



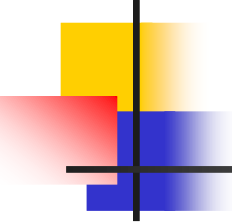
编译选项

- 内核组件
 - Y(*) 要集成该组件
 - N() 不需要该组件，以后会没有这项功能
 - M 以后再加该组件为一个外部模块



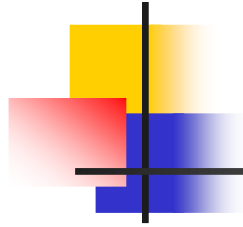
编译内核

- make
- make zImage
- make bzImage
- make modules



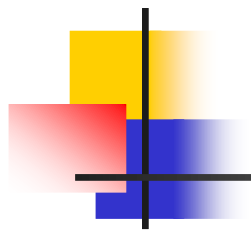
启用新内核

- make install□(慎用)
 - 将编译好的内核copy到/boot
- 配置引导菜单



```
## ## End Default Options ##
```

```
title      Ubuntu 8.04.2, kernel 2.6.24-23-generic
root       (hd0,0)
kernel     /boot/vmlinuz-2.6.24-23-generic root=UUID=
initrd     /boot/initrd.img-2.6.24-23-generic
quiet
```



```
title Win7
```

```
chainloader (hd0,0)/bootmgr
```

```
title win7
```

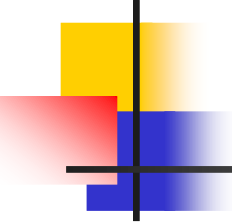
```
find --set-root /bootmgr
```

```
chainloader /bootmgr
```

```
title XP
```

```
find --set-root /ntldr
```

```
chainloader /ntldr
```



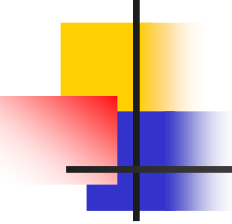
初始化程序的建立

- initrd

- mkinitrd /boot/initrd.img \$(uname -r)

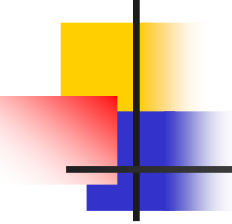
- initramfs

- mkinitramfs -o /boot/initrd.img 2.6.24-16
 - update-initramfs -u



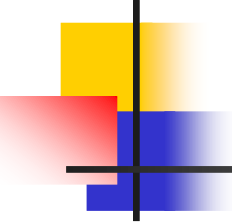
Debian和Ubuntu的简便办法

- `make-kpkg`
 - 用于`make menuconfig`之后
- 好处
 - 后面所有的部分自动做完
 - 会把编译好的内核打成**deb**安装包
 - 可以拷到其它机器安装



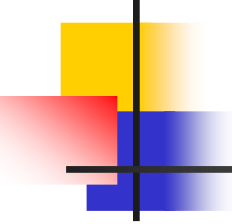
驱动

- 许多常见驱动的源代码集成在内核源码里
- 也有第三方开发的驱动，可以单独编译成模块.ko
- 编译需要内核头文件的支持



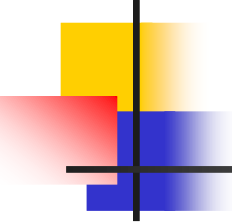
加载模块

- 底层命令
 - insmod
 - rmmod
- 高层命令
 - modprobe
 - modprobe -r



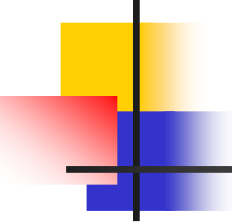
模块依赖

- 一个模块A引用另一个模块B所导出的符号，我们就说模块B被模块A引用。
- 如果要装载模块A，必须先要装载模块B。否则，模块B所导出的那些符号的引用就不可能被链接到模块A中。这种模块间的相互关系就叫做模块依赖。



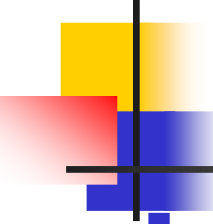
模块的依赖

- 自动按需加载
- 自动按需卸载
- moddep
- lsmod
- modinfo



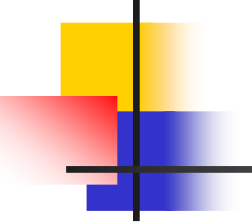
模块之间的通讯

- 模块是为了完成某种特定任务而设计的。其功能比较的单一，为了丰富系统的功能，所以模块之间常常进行通信。其之间可以共享变量，数据结构，也可以调用对方提供的功能函数。



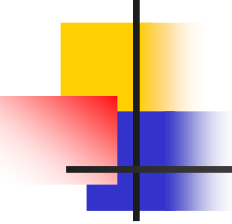
模块相关命令

- `insmod <module.ko> [module parameters]`
 - Load the module
 - 注意，只有超级用户才能使用这个命令
- `rmmod`
 - Unload the module
- `lsmod`
 - List all modules loaded into the kernel
 - 这个命令和`cat /proc/modules`等价
- `modprobe [-r] <module name>`
 - – Load the module specified and modules it depends



Linux内核模块与应用程序的区别

	C语言程序	Linux内核模块
运行	用户空间	内核空间
入口	main()	module_init() 指定;
出口	无	module_exit() 指定;
运行	直接运行	insmod
调试	gdb	kdebug, kdb, kgdb等



注意点

- 不能使用C库来开发驱动程序
- 没有内存保护机制
- 小内核栈
- 并发上的考虑

最简单的内核模块例子

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
static int __init hello_init(void)
{
    printk(KERN_INFO "Hello world\n");
    return 0;
}
static void __exit hello_exit(void)
{
    printk(KERN_INFO "Goodbye world\n");
}
module_init(hello_init);
module_exit(hello_exit);
```

- `static int __init hello_init(void)`
- `static void __exit hello_exit(void)`
 - **Static**声明，因为这种函数在特定文件之外没有其它意义
 - **__init**标记，该函数只在初始化期间使用。模块装载后，将该函数占用的内存空间释放
 - **__exit**标记 该代码仅用于模块卸载。
- **Init/exit**
 - 宏：**module_init/module_exit**
 - 声明模块初始化及清除函数所在的位置
 - 装载和卸载模块时，内核可以自动找到相应的函数

module_init(hello_init);
module_exit(hello_exit);

编译内核模块

- **Makefile文件**

```
obj-m := hello.o
```

```
all:
```

```
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules
```

```
clean:
```

```
    make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) clean
```

- **Module includes more files**

```
obj-m:=hello.o
```

```
hello-objs := a.o b.o
```


和硬件打交道

```
//file name: ioremap_driver.c
#include<linux/module.h>
#include<linux/init.h>
#include<asm/io.h>
//用于存放虚拟地址和物理地址
volatile unsigned long virt,phys;
//用与存放三个寄存器的地址
volatile unsigned long*GPBCON,*GPBDAT,*GPBUP;
void led_device_init(void){
    //0x56000010+0x10包揽全所有的IO引脚寄存器地址
    phys=0x56000010;
    // 0x56000010=GPBCON
    //在虚拟地址空间中申请一块长度为0x10的连续空间
    //这样，物理地址phys到phys+0x10对应虚拟地址
    virt到virt+0x10
    virt=(unsigned long)ioremap(phys,0x10);
```

```
void led_device_init(void)
{
    // 0x56000010 + 0x10 包揽全所有的IO引脚寄存器地址
    phys=0x56000010 ;
    // 0x56000010=GPBCON
    //在虚拟地址空间中申请一块长度为0x10的连续空间
    //这样，物理地址phys到phys+0x10对应虚拟地址
    virt到virt+0x10
    virt=(unsigned long)ioremap(phys,0x10);
    GPBCON=(unsigned long*)(virt+0x00);
    //指定需要操作的三个寄存器的地址
    GPBDAT=(unsigned long*)(virt+0x04);
    GPBUP=(unsigned long*)(virt+0x08);
}
//led配置函数,配置开发板的GPIO的寄存器
void led_configure(void)
{
```

```
}  
//led配置函数,配置开发板的GPIO的寄存器
```

```
void led_configure(void)  
{
```

```
    *GPBCON&=~ (3<<10) &~ (3<<12) &~ (3<<16) &~ (3<<20) ;  
    //GPB12 default 清零  
    *GPBCON|= (1<<10) | (1<<12) | (1<<16) | (1<<20) ;  
    //output 输出模式  
    *GPBUP|= (1<<5) | (1<<6) | (1<<8) | (1<<10) ;  
    //禁止上拉电阻  
}
```

```
//点亮led
```

```
void led_on(void)  
{
```

```
    *GPBDAT&=~ (1<<5) &~ (1<<6) &~ (1<<8) &~ (1<<10) ;  
}
```

```
//灭掉led
```

```
void led_off(void)
```

```
*GPBDAT&=~(1<<5) &~(1<<6) &~(1<<8) &~(1<<10);
}
//灭掉led
void led_off(void)
{
    *GPBDAT|=(1<<5) | (1<<6) | (1<<8) | (1<<10);
}
//模块初始化函数
static int __init led_init(void)
{
    led_device_init();
    //实现IO内存的映射
    led_configure();
    //配置GPB5 6 8 10为输出
    led_on();
    printk("hello ON!\n");
    return 0 ;
}
//模块卸载函数
static void __exit led_exit(void)
```

```
    led_on();  
    printk("hello ON!\n");  
    return 0 ;  
}  
//模块卸载函数  
static void __exit led_exit(void)  
{  
    led_off();  
    iounmap((void*)virt);  
    //撤销映射关系  
    printk("led OFF!\n");  
}  
module_init(led_init);  
module_exit(led_exit);  
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("hurryliu<>");  
MODULE_VERSION("2012-8-5.1.0");
```

模块参数传递

- 有些模块需要传递一些参数

- 参数在模块加载时传递

```
#insmod hello.ko test=2
```

- 参数需要使用module_param宏来声明

```
module_param(变量名称, 类型, 访问许可掩码)
```

- 支持的参数类型

```
Byte, short, ushort, int, uint, long, ulong, bool, charp  
Array (module_param_array(name, type, nump, perm))
```

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
#include <linux/moduleparam.h>
```

```
static int test;
module_param(test, int, 0644);
```

```
static int __init hello_init(void)
{
    printk(KERN_INFO "Hello world test=%d \n" , test);
    return 0;
}
static void __exit hello_exit(void)
{
    printk(KERN_INFO "Goodbye world\n");
}
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Test");
MODULE_AUTHOR("xxx");
module_init(hello_init);
module_exit(hello_exit);
```

模块之间的通讯实例

- 本实例通过两个模块来介绍模块之间的通信。模块`add_sub`提供了两个导出函数`add_integer()`和`sub_integer()`，分别完成两个数字的加法和减法。模块`test`用来调用模块`add_sub`提供的两个方法，完成加法或者减法操作。
- 1. `add_sub`模块
- 2. `test`模块
- 3. 编译模块


```
#ifndef _ADD_SUB_H_
#define _ADD_SUB_H_
long add_integer(long a, long b);
long sub_integer(long a, long b);
#endif
```

```
01 #include <linux/init.h>
02 #include <linux/module.h>
03 #include "add_sub.h"
04 long add_integer(int a, int b)
05 {
06     return a+b;
07 }
08 long sub_integer(int a, int b)
09 {
10     return a-b;
11 }
12 EXPORT_SYMBOL(add_integer);
13 EXPORT_SYMBOL(sub_integer);
14 MODULE_LICENSE("Dual BSD/GPL");
```

```
#include <linux/init.h>
#include <linux/module.h>
#include "add_sub.h"
```

/* 不要使用<>包含文件，否则找不到

```
/* 定义模块传递的参数 a, b */
```

```
static long a = 1;
```

```
static long b = 1;
```

```
static int AddOrSub = 1;
```

```
static int test_init(void)
```

/* 模块加载函数 */

```
{
```

```
    long result=0;
```

```
    printk(KERN_ALERT "test init\n");
```

```
    if(1==AddOrSub)
```

```
    {
```

```
        result=add_integer(a, b);
```

```
    }
```

```
    else
```

```
    {
```

```
        result=sub_integer(a, b);
```

```
    }
```

```
    printk(KERN_ALERT "The %s result is %ld",AddOrSub==1?"
```

```
    "Sub",result);
```

```
    return 0;
```

```
}
```

```
obj-m := test.o
KERNELDIR ?= /linux-2.6.29.4/linux-2.6.29.4
PWD := $(shell pwd)
SYMBOL_INC = $(obj)/../include
EXTRA_CFLAGS += -I $(SYMBOL_INC)
KBUILD_EXTRA_SYMBOLS=$(obj)/../print/Module.symvers
modules:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
modules_install:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules_install
clean:
    rm -rf *.o *~ core .depend *.cmd *.ko *.mod.c .tmp_versions
.PHONY: modules modules_install clean
```

导出符号表

- 如果一个模块需要向其他模块导出符号（方法或全局变量），需要使用：

`EXPORT_SYMBOL(name);`

`EXPORT_SYMBOL_GPL(name);`

*注意：符号必须在模块文件的全局部分导出，不能在函数部分导出。
更多信息可参考 `<linux/module.h>` 文件

- Modules 仅可以使用由 Kernel 或者其他 Modules 导出的符号 **不能使用 Libc**

- `/proc/kallsyms` 可以显示所有导出的符号

```
[root@localhost sample]# cat /proc/kallsyms
```

内核模块操作/proc文件

- **/proc文件系统**，这是**内核模块和系统交互**的两种主要方式之一。
- /proc文件系统也是Linux操作系统的特色之一。
- /proc文件系统不是普通意义上的文件系统，它是一个**伪文件系统**。
- 通过/proc，可以用标准Unix系统调用(比如**open()**、**read()**、**write()**、**ioctl()**等等)**访问进程地址空间**
- 可以用cat、more等命令查看/proc文件中的信息。
- 用户和应用程序可以通过/proc得到系统的信息，并可以改变内核的某些参数。
- 当调试程序或者试图获取指定进程状态的时候，/proc文件系统将是你强有力的支持者。通过它可以创建更强大的工具，获取更多信息。

/proc相关函数

- `create_proc_entry()` 创建一个文件
- `proc_symlink()` 创建符号链接
- `proc_mknod()` 创建设备文件
- `proc_mkdir()` 创建目录
- `remove_proc_entry()` 删除文件或目录

驱动类型

- Linux系统将设备分为3种类型：字符设备、块设备和网络接口设备。
 1. 字符设备 Character Driver
 2. 块设备 Block Driver
 3. 网络接口设备 Network Driver

简单的字符设备驱动程序

- 在Linux设备驱动程序的家族中，字符设备驱动程序是较为简单的驱动程序，同时也是应用非常广泛的驱动程序。所以学习字符设备驱动程序，对构建Linux设备驱动程序的知识结构非常的重要。

文件操作

——字符设备驱动的对上接口

- `ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);`
- `ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);`
- `int (*flush) (struct file *);`
- `int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);`
- `.....`

文件操作(file_operations)

——字符设备驱动的对上接口

```
struct file_operations scull_fops = {  
    .owner =      THIS_MODULE,  
    .llseek =     scull_llseek,  
    .read =       scull_read,  
    .write =      scull_write,  
    .ioctl =      scull_ioctl,  
    .open =       scull_open,  
    .release =    scull_release,  
};
```

两个基本结构

- file 结构体
- inode 结构体
 - dev_t i_rdev;
 - struct cdev *i_cdev;

字符设备驱动程序初始化加载过程

- 申请设备号
- 定义文件操作结构体 `file_operations`
- 创建并初始化定义结构体 `cdev`
- 将`cdev`注册到系统，并和对应的设备号绑定
- 在`/dev`文件系统中用`mknod`创建设备文件，并将该文件绑定到设备号上

主设备号和次设备号

- 一个字符设备或者块设备都有一个主设备号和次设备号。
- 主设备号和次设备号统称为设备号。
主设备号用来表示一个特定的驱动程序。
- 次设备号用来表示使用该驱动程序的设备。

申请和释放设备号

- `int register_chrdev_region(dev_t first, unsigned int count, char *name);`
- `int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count, char *name);`
- `void unregister_chrdev_region(dev_t first, unsigned int count);`

cdev结构体

- 在linux内核中使用cdev结构体来描述字符设备。该结构体是所有字符设备的抽象，其包含了大量字符设备所共有的特性。

cdev结构体的初始化

- `struct cdev *my_cdev = cdev_alloc();`
- `my_cdev->ops = &my_fops;`
- `void cdev_init(struct cdev *cdev, struct file_operations *fops);`

自定义结构

```
struct scull_dev {  
    struct scull_qset *data; /* Pointer to first quantum set */  
    int quantum;             /* the current quantum size */  
    int qset;                /* the current array size */  
    unsigned long size;      /* amount of data stored here */  
    unsigned int access_key; /* used by sculluid and scullpriv */  
    struct semaphore sem;    /* mutual exclusion semaphore */  
    struct cdev cdev;        /* Char device structure */  
};
```

设备注册

- 将设备注册到系统中:
- `int cdev_add(struct cdev *dev, dev_t num, unsigned int count);`
- 释放一个已经注册的设备:
- `void cdev_del(struct cdev *dev);`

实现file_operations中的各个函数

```
int scull_open(struct inode *inode, struct file *filp)
{
    struct scull_dev *dev; /* device information */

    dev = container_of(inode->i_cdev, struct scull_dev, cdev);
    filp->private_data = dev; /* for other methods */

    /* now trim to 0 the length of the device if open was write-only */
    if ( (filp->f_flags & O_ACCMODE) == O_WRONLY) {
        scull_trim(dev); /* ignore errors */
    }
    return 0;          /* success */
}
```

创建设备文件，并绑定到设备号

- 定义设备名device=scull
- 定义主设备号major=15
- 用户可以通过访问/dev/scull0来访问当前的驱动程序

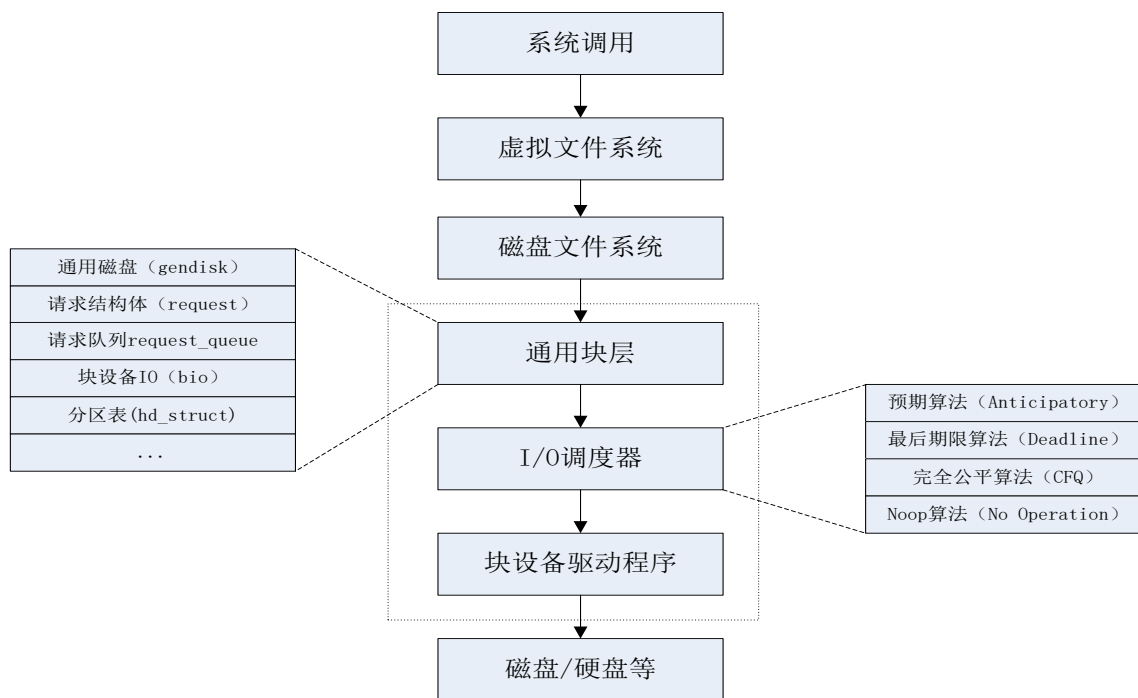
```
mknod /dev/${device}0 c $major 0
```

块设备驱动程序简介

- 除了字符设备、网络设备外，Linux系统中还有块设备。字符设备和块设备在内核中的结构有很大的不同，总体来说，块设备要比字符设备复杂很多。块设备主要包含磁盘设备、SD卡等，这些设备是Linux系统中不可缺少的存储设备。计算机中都需要这样的设备来存储数据，所以学会块设备驱动程序的写法是非常重要的。

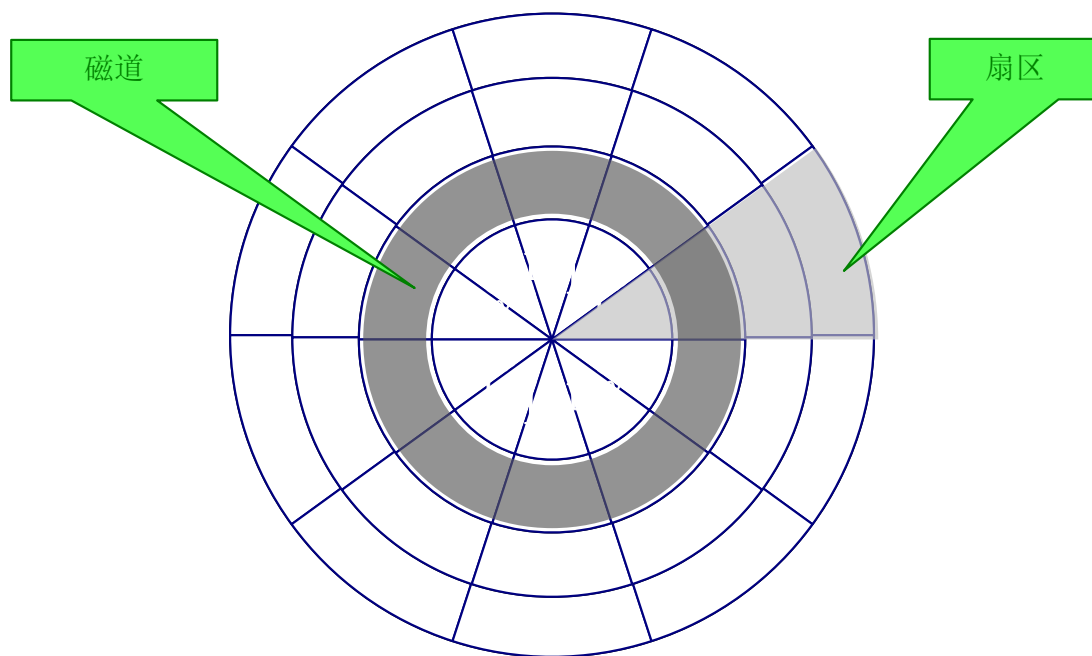
块设备总体概述

- Linux内核中，I/O设备大致分为两类：块设备和字符设备。块设备将信息存储在固定大小的块中，每个块都有自己的地址。数据块的大小通常在512字节到4K字节之间。块设备的基本特征是每个块都能独立于其它块而读写。磁盘就是最常见的块设备。在Linux内核中，块设备与内核其他模块的关系如图所示：



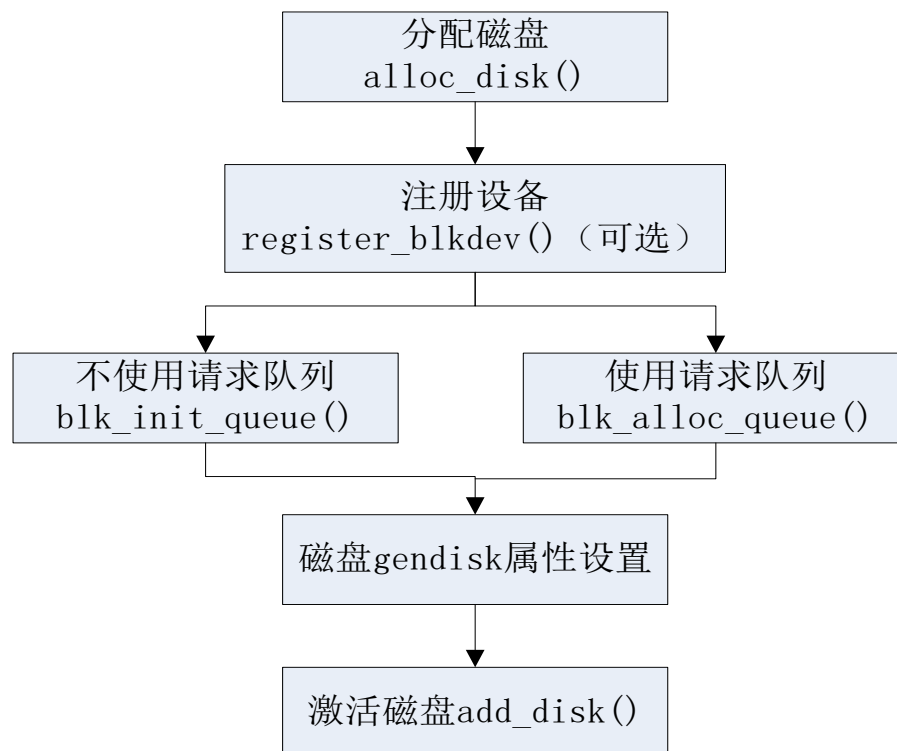
块设备的结构

- 在写块设备驱动程序之前，了解典型块设备的结构是非常重要的。图显示的是磁盘的一个盘面，一些重要的概念将在下面讲述。



块设备加载过程

- 在块设备的模块加载函数中，需要完成的一些重要工作，这些工作涉及到的一些重要概念，将在后面的小节中进行讲解，本节的目的目的是为了给出一个整体的概念。块设备驱动加载模块中需要完成的工作如下图所示：



块设备卸载过程

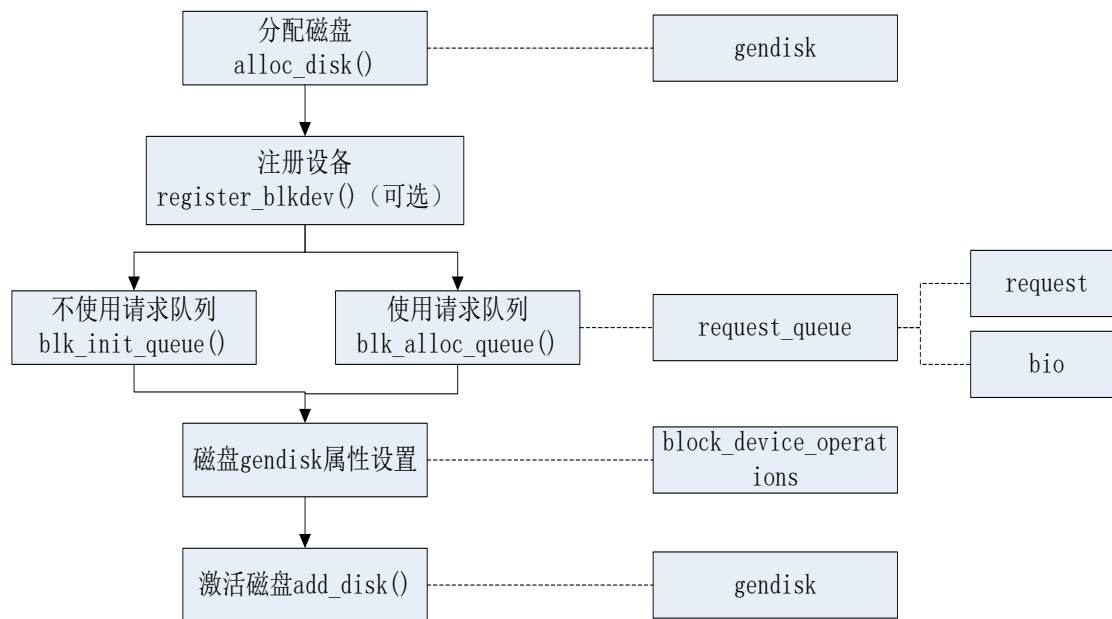
- 在块设备驱动的卸载模块中完成与模块加载函数相反的工作。
- （1）使用`del_gendisk()`函数删除gendisk设备，并使用`put_disk()`函数删除对gendisk设备的引用。
- （2）使用`blk_cleanup_queue()`函数清除请求队列，并释放请求队列所占用的资源。
- （3）如果在模块加载函数中使用了`register_blkdev()`注册设备，那么需要在模块卸载函数中使用`unregister_blkdev()`函数注销块设备，并释放对块设备的引用。

通用块层

- 通用块层是块设备驱动的核心部分，这部分主要包含块设备驱动程序的通用代码部分。本节将介绍通用块层的主要函数和数据结构。

通用块层

- 通用块层是一个内核组件，它处理来自系统其他组件发出的块设备请求。换句话说，通用块层包含了块设备操作的一些通用函数和数据结构。图是块设备加载函数中用到的一些重要数据结构，如通用磁盘结构gendisk、请求队列结构request_queue、请求结构request、块设备I/O操作结构bio、块设备操作结构block_device_operations等。



alloc_disk() 函数对应的 gendisk 结构体

- 现实生活中有许多具体的物理块设备，例如磁盘、光盘等。不同的物理块设备其结构是不一样的，为了将这些块设备公用属性在内核中统一，内核开发者定义了一个gendisk结构体来描述磁盘。gendisk是general disk的简称，一般称为通用磁盘。

块设备的注册和注销

- 为了使内核知道块设备的存在，需要使用块设备注册函数。在不使用块设备时，也需要注销块设备。块设备的注册和注销如下所述：
 - 1. 注册块设备函数register_blkdev()
 - 2. 注销块设备函数unregister_blkdev()

请求队列

- 简单的讲，一个块设备的请求队列就是包含块设备I/O请求的一个队列。这个队列使用链表线性的排列。请求队列中存储未完成的块设备I/O请求，并不是所有的I/O块请求都可以顺利的加入请求队列中。请求队列中定义了自己能处理的块设备请求限制。这些限制包括：请求的最大尺寸、一个请求能够包含的独立段数、硬盘扇区大小等。

设置gendisk属性中的 block_device_operations 结构体

- 在块设备中有一个和字符设备中file_operations对应的结构体block_device_operations。其也是一个对块设备操作的函数集合。
- 下面对这个结构体的主要成员进行分析。
- 1. 打开和释放函数
- 2. I/O控制函数
- 3. 介质改变函数
- 4. 使介质有效函数
- 5. 获得驱动器信息的函数
- 6. 模块指针

End

18.4.1 不使用请求队列的块设备驱动程序的组织

- 块设备函数驱动程序主要有一个加载函数、卸载函数和一个自定义的请求处理函数组成。本节将写一个虚拟的块设备驱动程序Virtual_blkdev。这个驱动程序在内存中开辟了一个8M的内存空间来模拟实际的物理块设备。这个块设备驱动程序代码比较简单，但功能却非常强大。对实际物理设备的操作命令同样可以应用在Virtual_blkdev这个块设备上，例如mkdir、mkesfs等命令。

18.4.2 宏定义和全局变量

- Virtual_blkdev块设备驱动中定义了一些重要的宏和全局指针，包括主设备号、设备名、设备的大小等。

18.4.3 加载函数

- Virtual_blkdev设备的加载函数主要完成分配磁盘、初始化请求队列、设置磁盘属性和激活磁盘的工作。

18.4.4 卸载函数

- Virtual_blkdev设备的卸载函数中主要完成与设备加载函数中相反的工作：
- （1）使用del_gendisk()函数删除gendisk设备。
- （2）使用put_disk()函数清楚gendisk的引用计数。
- （3）使用blk_cleanup_queue()函数清除请求队列。

18.4.5 自定义请求处理函数

- 内核将I/O读写请求放入请求结构request中，并连接到请求队列request_queue中。因为Virtual_blkdev设备是一个基于内存的设备，可以随机读取数据，并不需要复杂的I/O调度（I/O调度的作用是对请求结构request进行排序，最大限度的提高读写速率）。所以当请求到来时，将直接使用blk_init_queue()函数中注册的请求处理函数

Virtual_blkdev_do_request()函数，对请求

18.4.6 驱动测试

- 为了了解Virtual_blkdev这个块设备的特性，需要对其进行各方面的测试，这些测试如下所述。
- 1. 编译Virtual_blkdev.c文件
- 2. 加载模块文件
- 3. lsmod查看模块
- 4. 创建块设备文件
- 5. 在该设备上创建ext2文件系统
- 6. 挂载文件系统
- 7. 测试文件系统

18.5 I/O调度器

- Linux内核中，I/O调度器涉及到很多复杂的数据结构，而结构之间的关系又非常复杂。要精通这些知识，远非一章一节知识所能够达到。但本节力图给读者一个清晰的概念，随着内核的升级，这些概念可能有所细微的变化，但是其主要的原理是基本不会变化的。在详细讲解I/O调度器之前，需要知道数据是怎样从内存到达磁盘的。

18.5.1 数据从内存到磁盘的过程

- 内存是一个线性的结构，Linux系统将内存分为页。一页最大可以是64K，但是目前主流的系统页的大小都是4K。现在假设数据存储在内存的相邻几页中，希望将这些数据写到磁盘上。那么每一页的数据会被先封装为一个段，用bio_vec表示。多个页会被封装成多个段，这些段被组成以一个bio_vec为元素的数组，这个数组用bio_io_vec表示。

18.5.2 块I/O请求(bio)

- 数据从内存到磁盘或者从磁盘到内存的过程，叫做I/O操作。内核使用一个核心数据结构bio来描述I/O操作。
- 1. bio结构体
- bio结构体包含一个块设备完成一次I/O操作所需要的一切信息。
- 2. bio_vec结构体
- bio中的段用bio_vec结构体来表示。
- 3. bio结构体的相关宏
- 为了程序的可移植性，在写驱动程序时

18.5.3 请求结构 (request)

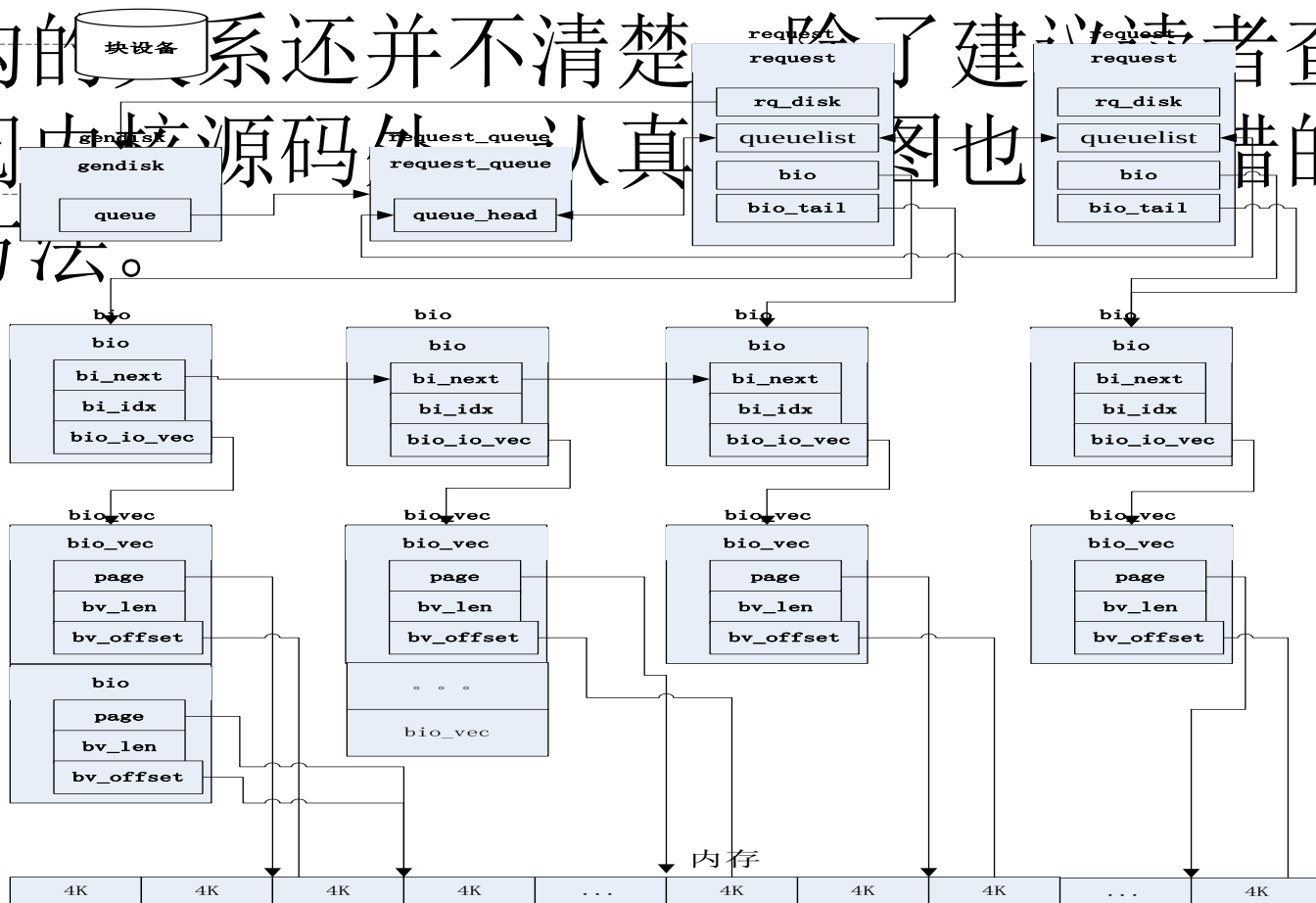
- 几个连续的页面会组成一个bio结构，几个相邻的bio结构就会组成一个请求结构request。这样当磁盘在接收一个与request对应的命令，就不需要大幅度的移动磁头，这样就节省了I/O操作的时间。

18.5.4 请求队列（ request_queue）

- 每个块设备驱动程序都维护着自己的请求队列request_queue，其包含设备将要处理的请求链表。请求队列主要用来连接对同一个块设备的多个request请求结构。同时请求队列中的一些字段还保存了块设备所支持的请求类型信息、请求的个数、段的大小、硬件扇区数等与设备相关的信息。总之，内核负责对请求队列的正确配置，使请求队列不会给块设备发送一个不能处理的请求。

18.5.5 请求队列、请求结构

- 可能读者对请求队列request_queue、请求结构request、bio等之间的关系、请求结构request、bio、bio_vec、gendisk等结构的系还并不清楚。除了建议读者查阅中核源码外，认真图也借的方法。



18.5.6 四种调度算法

- 对于像磁盘这样的块设备来说，是不能随机访问数据的。在访问实际的扇区数据以前，磁盘控制器必须花费很多时间来寻找扇区的位置，如果两个请求写操作在磁盘中的位置相离很远，那么写操作的大部分时间将花在寻找扇区上。所以内核需要提供一些调度方法，来使物理位置相邻的请求尽可能先后执行，那么就可以减少寻找扇区的时间，这种调度就叫做I/O调度。

18.6 自定义I/O调度器

- 本节接着上一节简介I/O调度器，并且仍然使用Virtual_blkdev设备，只是对其进行了一些简单的改进，使其效率更高。

18.6.1 Virtual_blkdev块设备的缺陷

- Virtual_blkdev块设备的数据都是存储在内存中的，对内存的访问可以随机进行，不需要对数据进行I/O调度。
18.4节中的Virtual_blkdev块设备使用了默认的I/O调度器。实际上，对于Virtual_blkdev来说，一个好的I/O调度器丝毫起不了一点作用，反而会浪费不少的CPU时间和内存。
- 出现这个问题的原因是因为I/O调度器的原理所致。I/O调度器试图合并一系

18.6.2 指定noop调度器

- linux内核中包含4个I/O调度器：Anticipatory、CFQ、Deadline和Noop。2.6.18之前的linux默认使用anticipatory，而之后的默认使用cfq。关于这4个调度器的原理已经在上一节做过介绍，这里不重复讲述。这里主要用到的是Noop调度器。Noop调度器是一个基本上不错任何事情的空调度器，它直接将I/O请求传递给通用块层，告诉通用块层已经对请求做了相应的调度处理。

18.6.3 Virtual_blkdev的改进实例

- 18.4节的Virtual_blkdev使用了默认的调度算法，但是并不符合内存设备的要求，这里对Virtual_blkdev_init()函数进行了简单的修改，使用noop调度算法取代了默认的调度算法。

18.6.4 编译和测试

- 本节对Virtual_blkdev的修改非常少，但是已经使Virtual_blkdev的效率提高了不少。使用 and 18.4节一样的编译方法编译新的Virtual_blkdev模块，make命令如下所示。

18.7 脱离I/O调度器

- 为了使读者详细的了解内核是怎么对数据进行读写的，这里对通用块层的函数调用关系进行了仔细分析。本节试图拜托繁琐的I/O调度器，对数据读写的本质进行分析，通过这种本质的学习读者将对数据读写的整个流程有一个深刻的理解。首先，将从请求队列中的bio处理函数开始。

18.7.1 请求队列中的bio处理函数

- 尽管上一节的noop调度器已经相当简单。它除了告诉内核一个bio已经调度完成，正在等待处理之外，几乎什么都不做。许多程序员错误的以为noop调度器的效率很高，是的，它确实比其他三种调度器效率要高，但是有比noop调度器效率更高的方法，那就是不用I/O调度器。

18.7.2 通用块层函数调用关系

- 有了上面关于请求队列的基础知识后，将分析一个块设备的读写过程。
- 1. 通用块层函数调用关系
- 2. 使用I/O调度器和不使用I/O调度器的分析
- 3. 使用I/O调度器和不使用I/O调度器的效率分析

18.7.3 对Virtual_blkdev块设备的改进

- 对Virtual_blkdev块设备的改进，首先需要修改Virtual_blkdev_init()函数，使其使用自定义的制造请求函数，修改后的代码如下所示：
 - 1. Virtual_blkdev_init()函数的修改
 - 2. 请求制造函数
Virtual_blkdev_make_request()

18.7.4 编译和测试

- 使用make命令对新的Virtual_blkdev块设备进行编译，命令如下。

18.8 块设备的物理结构

- 上几节介绍的块设备，其基本功能还不完善。本节将块设备的物理结构进行完善，首先介绍分区。

18.8.1 为Virtual_blkdev块设备添加分区

- 对于实际的物理磁盘，一般都有多个分区，本节将对Virtual_blkdev设备进行分区，分区的概念是：
 1. 分区
- 分区是物理磁盘的一部分，将物理磁盘分为几个单独的单元，每一个单元就是一个分区。每一个分区可以用来存放文件和数据。
- 2. alloc_disk()函数增加分区
- 比起I/O调度来，对磁盘进行分区则非常容易，因为内核做了大部分的工作。这些工作大都由fs/partitions目录中的文件来完成。

18.8.2 对新的Virtual_blkdev 代码的分析

- 只要对18.7节中的代码做一点简单的修改，就能够使设备支持分区。首先在文件开始添加一个分区数目的宏，代码如下所示。

18.8.3 编译和测试

- 在使用Virtual_blkdev设备前，需要先编译和调试代码，这些步骤如下所示：
 - 1. 编译代码
 - 2. 加载模块
 - 3. 分区
 - 4. 测试分区数

18.8.4 分区数的计算

- 一个磁盘的最大分区数目由两方面决定，第一是`alloc_gendisk()`函数中指定的最大分区数，第二是磁盘的物理磁道数。在上述的`Virtual_blkdev`块设备的代码中，并没有为磁盘指定磁道数，这样情况下，Linux内核只能猜测磁盘的磁道数。

18.8.5 设置Virtual_blkdev的结构

- Virtual_blkdev是一个内存设备，对于内存设备来说，其物理结构并不十分重要。驱动开发人员可以根据需要对其进行设置，这些设置包括磁盘盘面数、扇区数、磁道数等。如果对于实际的物理设备，那么就不能对其物理结构顺便设置，因为这些物理结构在硬件出厂时，就已经固定了。

18.8.6 编译和测试

- 在使用Virtual_blkdev设备前，需要先编译和调试代码，这些步骤如下所示：
 - 1. 编译代码
 - 2. 加载模块
 - 3. 分区
 - 4. 查看分区
 - 5. 创建文件系统
 - 6. 挂载文件系统

18.9 小结

- 块设备的操作与字符设备非常不同，在本章中介绍了大量的与块设备相关的数据结构，例如request_queue、request、bio等。在本章中反复的讲到请求，请求是完成块设备读写操作的基本单位。
- 从块设备驱动整体代码构架来看，请求队列是必须的。根据块设备的不同物理结构，请求队列可以使用系统提供的“制造请求”函数，也可能使用自定义的“请求制造”函数。自定义的“请求制造”函数有更高的效率，当缺少通用性，只能针对特定的块设备。
- 如果读者从本章开始一直读来，那么会发现Virtual_blkdev块设备最开始有很多缺陷，随着讲述的深入，这种缺陷慢慢减少，越来越像实际的物理块设备了。在使用fdisk对块设备分区时，使用了block_device_operations结构体及相应的getgeo()函数，使块设备支持更多的分区。